Optimización de Consultas en PostgreSQL

Optimización mediante índices

Con el objetivo de mejorar el rendimiento de las consultas más costosas del sistema, se implementaron varios índices sobre las tablas con mayor volumen de datos y sobre las columnas utilizadas con mayor frecuencia en filtros (WHERE), uniones (JOIN) y ordenamientos (ORDER BY).

Los índices creados fueron los siguientes:

Índice	Tabla	Objetivo
idx_trips_composite_joins	trips	Optimizar los JOIN entre viajes, vehículos, conductores y rutas para consultas sobre viajes completados.
idx_trips_departure	trips	Mejorar las búsquedas por rangos de fechas utilizando departure_datetime.
idx_deliveries_scheduled_datetime	deliveries	Optimizar consultas que filtran entregas realizadas por fecha programada.
idx_deliveries_status	deliveries	Acelerar consultas que filtran por estado de la entrega.
idx_maintenance_vehicle_cost	maintenance	Optimizar la búsqueda de mantenimientos asociados a cada vehículo.
idx_drivers_status_license	drivers	Mejorar las consultas sobre conductores activos y vencimiento de licencias.
idx_routes_metrics	routes	Facilitar el acceso a información de rutas utilizada en cálculos de distancia y destino.
idx_vehicles_status	vehicles	Optimizar filtros sobre el estado de los vehículos.

Es importante mencionar que la creación de un índice no implica que PostgreSQL lo utilice en todas las consultas. El optimizador del motor de base de datos evalúa diferentes estrategias de ejecución y selecciona aquella cuyo costo estimado sea menor.

En los casos donde el plan de ejecución muestra un Sequential Scan (Seq Scan), esto generalmente ocurre por una de las siguientes razones:
La tabla contiene pocos registros y recorrerla completamente resulta más económico que acceder mediante un índice.
La consulta necesita procesar una gran parte de la tabla, por lo que el índice aporta poco beneficio.
La condición del filtro utiliza funciones o expresiones que impiden aprovechar directamente el índice.
El mayor costo de la consulta se encuentra en operaciones posteriores como JOIN, GROUP BY, ORDER BY o funciones de ventana, y no en la búsqueda inicial de registros.

A continuación, se presenta el análisis individual de cada consulta utilizando los resultados obtenidos con EXPLAIN ANALYZE.

Query 1: Contar vehículos por tipo
Problema de negocio: Conocer la composición de la flota

Consulta 1 – Cantidad de vehículos por tipo
Tiempo de ejecución: 0.295 ms

Plan utilizado: Seq Scan + HashAggregate

Aunque existe un índice sobre el estado de los vehículos (idx_vehicles_status), esta consulta no filtra por dicha columna y únicamente necesita recorrer los 200 registros de la tabla para agruparlos por tipo. Debido al tamaño reducido de la tabla, PostgreSQL determina que un recorrido secuencial es la alternativa más eficiente.

Query 2: Listar conductores con licencia próxima a vencer
Problema de negocio: Prevenir problemas legales por licencias vencidas

Consulta 2 – Licencias próximas a vencer
Tiempo de ejecución: 0.344 ms

Plan utilizado: Seq Scan

Se creó el índice idx_drivers_status_license para optimizar búsquedas sobre conductores activos y fechas de vencimiento de licencia. Sin embargo, esta consulta únicamente filtra por license_expiry y no por status, condición necesaria para aprovechar el índice parcial. Además, la tabla contiene únicamente 400 registros, por lo que PostgreSQL opta por un recorrido secuencial.

Query 3: Total de viajes por estado
Problema de negocio: Monitorear operaciones en curso

Consulta 3 – Cantidad de viajes por estado
Tiempo de ejecución: 52.154 ms

Plan utilizado: Seq Scan + HashAggregate

Aunque existen índices sobre la tabla trips, la consulta necesita leer los 100.000 registros para agruparlos por estado. Como prácticamente toda la tabla debe procesarse, un índice no representa una mejora significativa y PostgreSQL selecciona un recorrido secuencial.

Query 4: Total de entregas por ciudad destino en los últimos 2 meses
Problema de negocio: Identificar demanda por ciudad para planificación de recursos

Consulta 4 – Ciudades con mayor número de entregas
Tiempo de ejecución: 181.751 ms

Plan utilizado: Bitmap Index Scan

En esta consulta PostgreSQL aprovecha el índice idx_trips_departure, utilizando un Bitmap Index Scan para recuperar únicamente los viajes correspondientes a los últimos 60 días. Posteriormente realiza los JOIN con las tablas deliveries y routes. Aunque también existen índices sobre rutas y viajes, el optimizador determina que el índice sobre la fecha es el que ofrece la mayor reducción de registros.

Query 5: Conductores activos con cantidad de viajes completados
Problema de negocio: Evaluar carga de trabajo por conductor

Consulta 5 – Rendimiento de conductores activos
Tiempo de ejecución: 160.908 ms

Plan utilizado: Seq Scan

Aunque existe el índice idx_drivers_status_license, la tabla drivers es pequeña y la consulta debe recorrer completamente la tabla trips para calcular las métricas de desempeño. El costo principal se encuentra en las agregaciones sobre los viajes y no en la búsqueda de conductores, por lo que PostgreSQL no considera necesario utilizar el índice.

Query 6: Promedio de entregas por conductor en los últimos 6 meses
Problema de negocio: Medir productividad individual de conductores

Consulta 6 – Productividad por conductor
Tiempo de ejecución: 213.548 ms

Plan utilizado: Bitmap Index Scan

El índice idx_trips_departure es utilizado para recuperar únicamente los viajes realizados durante los últimos seis meses. Esta reducción inicial permite disminuir el volumen de información antes de ejecutar las uniones y agrupaciones necesarias para calcular la productividad de cada conductor.

Query 7: Rutas con mayor consumo de combustible por kilómetro
Problema de negocio: Identificar rutas ineficientes para optimización

Consulta 7 – Consumo promedio de combustible por ruta
Tiempo de ejecución: 169.996 ms

Plan utilizado: Parallel Seq Scan

Aunque se creó el índice idx_routes_metrics, la mayor carga de trabajo corresponde al procesamiento de la tabla trips, donde una gran cantidad de registros cumplen las condiciones del filtro. En este escenario PostgreSQL obtiene un mejor rendimiento distribuyendo el recorrido secuencial entre varios procesos (Parallel Seq Scan) que utilizando un índice.

Query 8: Análisis de entregas retrasadas por día de la semana
Problema de negocio: Identificar patrones de retraso para mejorar planificación

Consulta 8 – Entregas por día de la semana
Tiempo de ejecución: 64.293 ms

Plan utilizado: Bitmap Index Scan

La consulta aprovecha el índice idx_deliveries_scheduled_datetime, recuperando unicamente las entregas realizadas durante los últimos 90 días antes de efectuar la agrupación por día de la semana.

Query 9: Costo de mantenimiento por kilómetro recorrido
Problema de negocio: Evaluar costo-beneficio de cada tipo de vehículo

Consulta 9 – Costo de mantenimiento por kilómetro
Tiempo de ejecución: 2939.531 ms

Plan utilizado: Index Scan

Esta consulta utiliza el índice idx_maintenance_vehicle_cost para acceder eficientemente a los registros de mantenimiento de cada vehículo. No obstante, sigue siendo la consulta más costosa debido al elevado número de JOIN, agregaciones y cálculos requeridos, además del gran volumen de datos procesados.

Query 10: Ranking de conductores por eficiencia usando Window Functions
Problema de negocio: Identificar top performers para incentivos

Consulta 10 – Ranking de conductores
Tiempo de ejecución: 230.521 ms

Plan utilizado: Bitmap Index Scan

Se utiliza el índice idx_trips_departure para limitar el análisis a los viajes de los últimos tres meses. El mayor tiempo de ejecución proviene posteriormente de las funciones de ventana (Window Functions) empleadas para calcular el ranking de los conductores.

Query 11: Análisis de tendencia de viajes con LAG y LEAD
Problema de negocio: Proyectar necesidades futuras basadas en tendencias

Consulta 11 – Tendencia mensual de viajes completados
Tiempo de ejecución: 146.630 ms

Plan utilizado: Seq Scan

Aunque existe el índice compuesto idx_trips_composite_joins, este está diseñado principalmente para optimizar uniones entre tablas y no consultas que agrupan únicamente por fecha. Además, aproximadamente el 90 % de los viajes corresponden al estado completed, por lo que PostgreSQL considera más eficiente recorrer toda la tabla que utilizar un índice con baja selectividad.

Query 12: Pivot de entregas por hora y día de la semana
Problema de negocio: Optimizar horarios de operación y personal

Consulta 12 – Distribución de entregas por hora
Tiempo de ejecución: 266.584 ms

Plan utilizado: Parallel Seq Scan

Aunque existe el índice idx_deliveries_scheduled_datetime, la consulta aplica la función EXTRACT(HOUR FROM scheduled_datetime). Al utilizar una función sobre la columna indexada, PostgreSQL no puede aprovechar directamente el índice para esa parte del filtro y decide realizar un recorrido secuencial paralelo, que resulta más eficiente para este tipo de procesamiento.

Consulta	Tiempo sin índices (ms)	Tiempo con índices (ms)	Diferencia (ms)	Mejora (%)	Observación
1	0.191	0.295	+0.104	**-54.5%**	Sin cambios. La tabla tiene solo 200 registros, el optimizador mantiene Seq Scan.
2	0.141	0.344	+0.203	**-144.0%**	También es una tabla pequeña; el índice no aporta beneficio.
3	35.274	52.154	+16.880	**-47.9%**	Consulta agregada sobre toda la tabla; debe leer todos los registros.
4	213.325	181.751	-31.574	**14.8%**	El índice sobre fecha reduce el conjunto de viajes antes del JOIN.

5	69.153	160.908	+91.755	**-132.7%**	El índice no ayuda porque la consulta sigue leyendo toda la tabla Trips.
6	153.992	213.548	+59.556	-38.7%	El filtro usa índice, pero el costo de JOIN y GROUP BY domina el tiempo.
7	188.014	169.996	-18.018	9.6%	Ligera mejora gracias al índice sobre fechas.
8	145.466	64.293	-81.173	55.8%	Gran mejora. Se reemplaza Seq Scan por Bitmap Index Scan sobre scheduled_datetime.
9	12130.860	2939.531	-9191.329	75.8%	Mayor mejora del proyecto. El índice elimina millones de comparaciones y usa Index Scan + Memoize.
10	163.943	230.521	+66.578	-40.6%	El índice filtra pocos registros; el costo principal sigue siendo la agregación.
11	148.604	146.630	-1.974	1.3%	Diferencia mínima; la ordenación sigue siendo el cuello de botella.
12	262.057	266.584	+4.527	-1.7%	Sin cambios significativos. Continúa usando Parallel Seq Scan.

La comparación demuestra que los índices son más efectivos cuando la consulta realiza búsquedas selectivas sobre columnas utilizadas en WHERE, JOIN o ORDER BY. Cuando una consulta necesita procesar la mayor parte de la tabla o ejecutar agregaciones complejas, el optimizador suele preferir un Sequential Scan, ya que el costo adicional de acceder al índice supera el beneficio obtenido. Por ello, la creación de índices debe realizarse con base en el patrón de consultas y no de manera indiscriminada.